

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 9 | Part 1

Optimization Problems and Design Strategies

Optimization Problems

- ▶ We often want to find the **best**.
 - ▶ Shortest path between two nodes.
 - ▶ Minimum spanning tree.
 - ▶ Schedule that maximizes tasks completed.
 - ▶ Best recommendation.
 - ▶ Line of best fit.
- ▶ These are **optimization problems**.

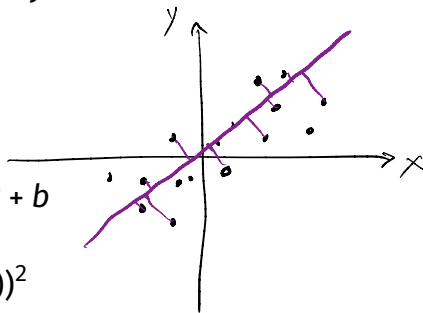
Example: Regression

- ▶ Given a set of n points in $\mathbb{R}^2$, find a straight line $y = mx + b$ which minimizes the Sum of Squared Errors.

- ▶ **Given:** set of n points $\{(x_i, y_i)\}$ in $\mathbb{R}^2$

- ▶ **Search Space:** all straight lines of form $y = mx + b$

- ▶ **Objective Function:** $\phi(m, b) = \sum_{i=1}^n (y_i - (mx_i + b))^2$



Continuous Optimization

- ▶ Here, the search space is continuous, often **infinite**.
- ▶ Methods for solving often use calculus.

Discrete Optimization

- ▶ Here, the search space is discrete, typically **finite**.
- ▶ Example: shortest path between two nodes.
- ▶ Methods for solving (usually) can't use calculus.
- ▶ We will focus on these problems.

Brute Force

- ▶ If search space is finite, can employ **brute force search**.
- ▶ Typically search space is too large to be feasible.

Design Strategies

- ▶ Focus on **design strategies** for discrete optimization.:
 - ▶ **Divide-and-conquer algorithms**:
 - ▶ Binary search. Mergesort, Quicksort, Median and order statistics.
 - ▶ **Greedy Algorithms**
 - ▶ **Backtracking**
 - ▶ **Dynamic Programming**

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 9 | Part 2

Greedy Algorithms

Main Idea

For some problems, a greedy approach is guaranteed to find the optimal solution. For other problems, it is not.

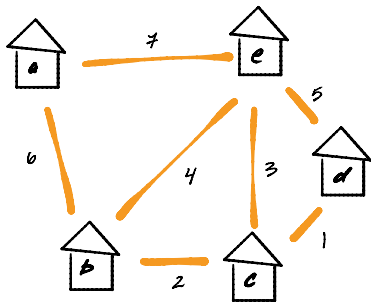
Main Idea

Coming up with a greedy algorithm is usually simple – proving that it finds the optimal may not be so easy.

Today

- ▶ Minimum Spanning Tree
- ▶ Maximum Coverage: applications in facility location, viral marketing, etc.

Minimum Spanning Tree



- ▶ Choose a set of power lines to build so that:
 - ▶ electricity can reach every house;
 - ▶ total cost of the lines is minimized.
- ▶ Solution: compute a **minimum spanning tree**.

MSTs in Data Science?

- ▶ Do we need to find MSTs in data science?
- ▶ Actually, yes! (Clustering)

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 9 | Part 3

Prim's Algorithm

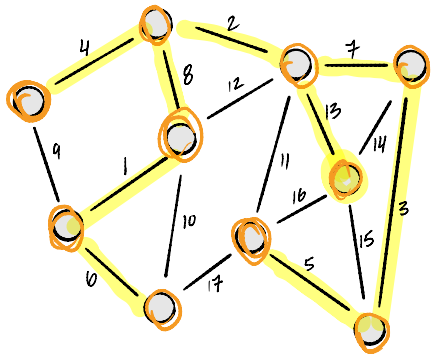
Building MSTs

- ▶ How do we build a MST efficiently?
- ▶ We'll adopt a **greedy** approach.
 - ▶ Build a tree edge-by-edge.
 - ▶ At every step, doing what looks best at the moment.
- ▶ This strategy isn't guaranteed to work in all of life's situations, but it works for building MSTs.

Two Greedy Approaches

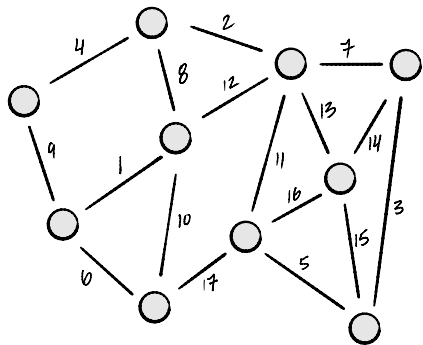
- ▶ We'll look at two greedy algorithms:
 - ▶ Today: Prim's Algorithm
 - ▶ ~~Prim's~~ Kruskal's Algorithm
- ▶ Differ in the order in which edges are added to tree.
- ▶ Also differ in time complexity.

Prim's Algorithm, Informally



- ▶ Start by picking any node to add to “tree”, T .
- ▶ While T is not a spanning tree, greedily add **lightest** edge from a node in T to a node not in T .
 - ▶ “lightest” = edge of smallest weight

Prim's Algorithm, Informally

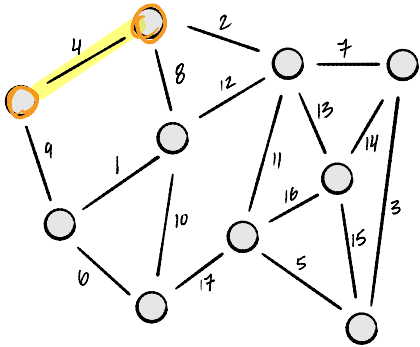


- ▶ Start by picking any node to add to “tree”, T .
- ▶ While T is not a spanning tree, greedily add **lightest** edge from a node in T to a node not in T .
 - ▶ “lightest” = edge of smallest weight
- ▶ Is this guaranteed to work? Yes, as we'll see.

Prim's Algorithm, Equivalently

- ▶ For each node u , store:
 - ▶ estimated cost of adding node to tree;
 - ▶ estimated “predecessor” v in the tree.
- ▶ At each step,
 - ▶ Find node with smallest estimated cost.
 - ▶ Add to tree T by including edge with estimated “predecessor”.
 - ▶ Update cost of neighbors.
- ▶ Same as adding lightest edge from T to outside T at every step!

Prim's Algorithm, Equivalently



spanning tree

- ▶ While T is not a tree:
 - ▶ find the node $u \notin T$ with smallest cost
 - ▶ add the edge between u and its estimated “predecessor” to T
 - ▶ update estimated cost/pred. of u ’s neighbors which aren’t already in tree.

Recall: Priority Queues

- ▶ How do we efficiently find node with smallest cost?
- ▶ Priority Queues:
 - ▶ `PriorityQueue(priorities)`: creates priority queue from dictionary whose values are priorities.
 - ▶ `.extract_min()`: removes and returns key with smallest value.
 - ▶ `.decrease_priority(key, value)`: changes key's value.
- ▶ We'll use a priority queue to hold nodes not yet added to tree.

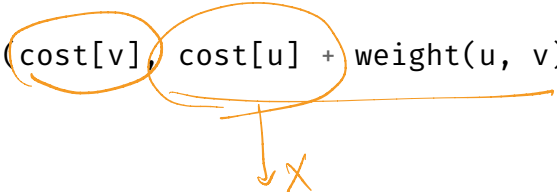
```
def prim(graph, weight):
    tree = UndirectedGraph()

    estimated_predecessor = {node: None for node in graph.nodes}
    cost = {node: float('inf') for node in graph.nodes}
    priority_queue = PriorityQueue(cost)

    while priority_queue:
        u = priority_queue.extract_min()
        if estimated_predecessor[u] is not None:
            tree.add_edge(estimated_predecessor[u], u)
        for v in graph.neighbors(u):
            if weight(u, v) < cost[v] and v not in tree.nodes:
                priority_queue.decrease_priority(v, weight(u, v))
                cost[v] = weight(u, v)
                estimated_predecessor[v] = u
    return tree
```

Prim and Dijkstra

- ▶ This is a lot like Dijkstra's Algorithm for s.p.d.!
- ▶ Both: at each step, extract node with smallest cost, update its edges.
(Prim: only those edges to nodes not in tree).
- ▶ Dijkstra update of (u, v) :

$$\text{cost}[v] = \min(\text{cost}[v], \text{cost}[u] + \text{weight}(u, v))$$


- ▶ Prim update of (u, v) :

$$\text{cost}[v] = \min(\text{cost}[v], \text{weight}(u, v))$$

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 9 | Part 4

Time Complexity

Time Complexity

- ▶ A priority queue can be implemented using a **heap**.
- ▶ If a **binary min-heap** is used:
 - ▶ `PriorityQueue(est)` takes $\Theta(V)$ time.
 - ▶ `.extract_min()` takes $O(\log V)$ time.
 - ▶ `.decrease_priority()` takes $O(\log V)$ time.

Time Complexity

```
def prim(graph, weight):  
    tree = UndirectedGraph()
```

```
    estimated_predecessor = {node: None for node in graph.nodes}  
    cost = {node: float('inf') for node in graph.nodes}  
    priority_queue = PriorityQueue(cost)  $\longrightarrow \Theta(V)$ 
```

```
    while priority_queue:  
        u = priority_queue.extract_min()  $\longrightarrow \log(V)$   
        if estimated_predecessor[u] is not None:  
            tree.add_edge(estimated_predecessor[u], u)  
        for v in graph.neighbors(u):  
            if weight(u, v) < cost[v] and v not in tree.nodes:  
                priority_queue.decrease_priority(v, weight(u, v))  $\rightarrow \log(V)$   
                cost[v] = weight(u, v)  
                estimated_predecessor[v] = u
```

```
    return tree
```

Total: $\Theta(V \log V + E \log V)$

Time Complexity

- ▶ Using a **binary heap**...
- ▶ Overall: $\Theta(V \log V + E \log V)$.
- ▶ Since graph is assumed connected, $E = \Omega(V)$.
- ▶ So this simplifies to $\Theta(E \log V)$.

Fibonacci Heaps

- ▶ A priority queue can be implemented using a **heap**.
- ▶ If a **Fibonacci min-heap** is used:
 - ▶ `PriorityQueue(est)` takes $\Theta(V)$ time.
 - ▶ `.extract_min()` takes $\Theta(\log V)$ time¹.
 - ▶ `.decrease_priority()` takes $O(1)$ time.

¹Amortized.

Time Complexity

```
def prim(graph, weight):  
    tree = UndirectedGraph()  
  
    estimated_predecessor = {node: None for node in graph.nodes}  
    cost = {node: float('inf') for node in graph.nodes}  
    priority_queue = PriorityQueue(cost)
```

```
    while priority_queue:  
        u = priority_queue.extract_min()  $\longrightarrow \log V$   
        if estimated_predecessor[u] is not None:  
            tree.add_edge(estimated_predecessor[u], u)  
        for v in graph.neighbors(u):  
            if weight(u, v) < cost[v] and v not in tree.nodes:  
                priority_queue.decrease_priority(v, weight(u, v))  $\rightarrow O(1)$   
                cost[v] = weight(u, v)  
                estimated_predecessor[v] = u  
    return tree
```

$\Theta(V \log V + E)$

Time Complexity

- ▶ Using a **Fibonacci heap**...
- ▶ Overall: $\Theta(V \log V + E)$.

Fibonacci vs. Binary Heaps

- ▶ Using Fibonacci heaps improves time complexity when graph is dense.
- ▶ E.g., if $E = \Theta(V^2)$:
 - ▶ Prim's with Fibonacci: $\Theta(E) = \Theta(V^2)$
 - ▶ Prim's with binary: $\Theta(E \log E) = \Theta(V^2 \log V)$.
- ▶ But Fibonacci heaps are **hard** to implement; have large constants.
- ▶ Binary heaps used more in practice despite complexity.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 9 | Part 5

Correctness of Prim's Algorithm

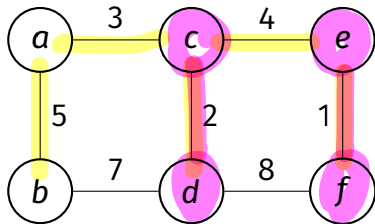
Being Greedy

- ▶ At every step, we add the lightest edge.
- ▶ Is this “safe”?

Being Greedy

- ▶ At every step, we add the lightest edge.
- ▶ Is this “safe”?
- ▶ Yes! This is guaranteed to find an MST.

Promising Subtrees

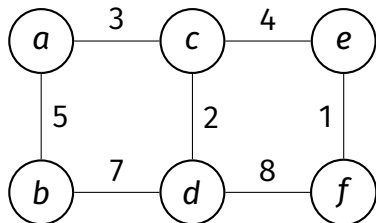


- ▶ Let $G = (V, E, \omega)$ be a weighted graph.
- ▶ A subgraph $T' = (V', E')$ is **promising** if it is “part” of some MST.
 - ▶ That is, it is an “MST in progress”
 - ▶ Not necessarily a tree!
- ▶ That is, there exists an MST $T = (V, E_{\text{mst}})$ such that $E' \subset E_{\text{mst}}$.
- ▶ Hint: a “promising subtree” where $V' = V$ is an MST!

Main Idea

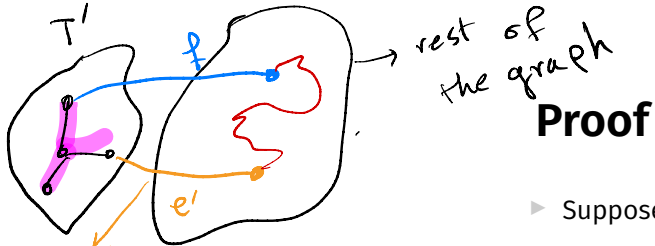
Prim's starts with a promising subtree T . At each step, adds lightest edge from a node within T to a node outside of T .

We'll show each new edge results in a larger promising subtree. Eventually the promising subtree becomes a full MST.



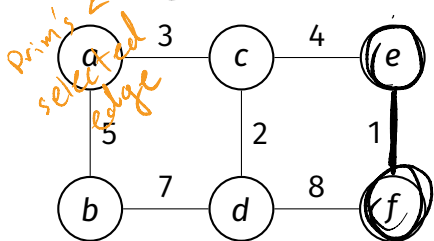
Claim

- ▶ Let $G = (V, E, \omega)$ be a weighted graph.
- ▶ Suppose $T' = (V', E')$ is a promising subtree for an MST of G .
- ▶ Let $e = (u, v)$ be a **lightest edge** from a node in T' to a node outside of T' . (Prim).
- ▶ Then adding (u, v) to T' results in another **promising subtree**.



Proof

- ▶ Suppose T_{mst} is an MST that includes T' .
- ▶ If T_{mst} includes e , we're done: $T' + e$ is promising.
- ▶ If it doesn't include e , it must have an edge f that connects T' to rest of the graph.
- ▶ Swap f with e in T_{mst} . The result is a tree, and it must be a MST since $\omega(e) \leq \omega(f)$.
- ▶ So there is an MST that contains $T' + e$.



CS-GY 6033

Design and Analysis of Algorithms I

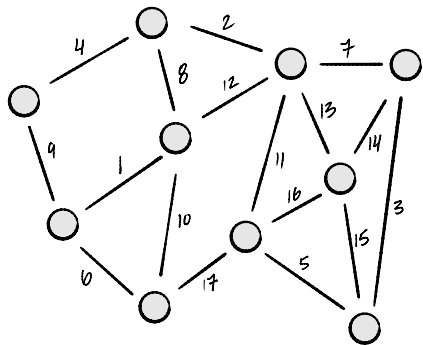
Lecture 9 | Part 6

Kruskal's Algorithm

Two Greedy Approaches

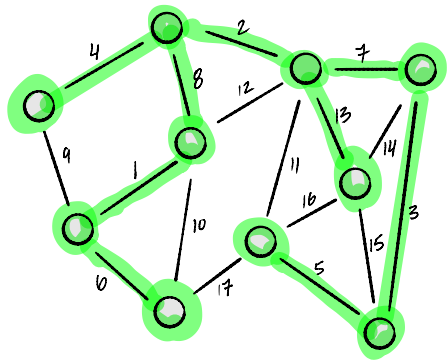
- ▶ We'll look at two greedy algorithms:
 - ▶ Prim's Algorithm
 - ▶ Kruskal's Algorithm
- ▶ Differ in the order in which edges are added to tree.
- ▶ Also differ in time complexity.

Prim's Algorithm, Informally



- Start by picking any node to add to “tree”, T .
- While T is not a spanning tree, greedily add **lightest** edge from a node in T to a node not in T .
 - “lightest” = edge of smallest weight

Kruskal's Algorithm, Informally



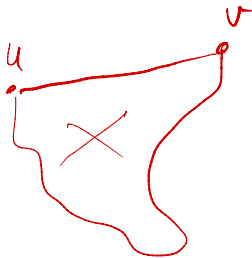
- ▶ Start with empty forest: $T = (V, E_{\text{mst}})$, where $E_{\text{mst}} = \emptyset$.
- ▶ Loop through edges in increasing order of weight.
 - ▶ If edge does not create a cycle in T , add it to T .
 - ▶ If T is a spanning tree, break.

Being Greedy

- ▶ Prim: add the **node** with smallest estimated cost and update neighbors.
 - ▶ Works locally, “grows” a connected tree.
- ▶ Kruskal: add the **edge** with smallest weight.
 - ▶ As long as it doesn't make a cycle.
 - ▶ Edge can be anywhere in graph.

Kruskal's Algorithm (Pseudocode)

```
def kruskal(graph, weights):  
    mst = UndirectedGraph()  
  
    # sort edges in ascending order by weight  
    sorted_edges = sorted(graph.edges, key=weights)  
  
    for (u, v) in sorted_edges:  
        # if u and v are not already connected  
        if ...:  
            mst.add_edge(u, v)  
  
        # (optional) if mst is now a spanning tree, break  
        if len(mst.edges) == len(graph.nodes) - 1:  
            break  
  
    return mst
```



Checking for Connectivity

- ▶ Each iteration: check if u and v are connected in $T = (V, E_{\text{mst}})$.
- ▶ We *could* do a DFS/BFS on each iteration...
 - ▶ $\Theta(V + E_{\text{mst}}) = \Theta(V)$ each time.
 - ▶ **Expensive!**
- ▶ Remember:
 - ▶ If you're computing something once, use a fast algorithm.
 - ▶ If you're computing it repeatedly, consider a **data structure**.

Disjoint Set Forests

- ▶ Represent a collection of disjoint sets.

$\{\{1, 5, 6\}, \{2, 3\}, \{0\}, \{4\}\}$

- ▶ `.union(x, y)`: Union the sets containing `x` and `y`.
- ▶ `.in_same_set(x, y)`: Return **True/False** if `x` and `y` are in the same set.²

²Usually implemented as a `.find(x)` method returning representative of set containing `x`.

Example

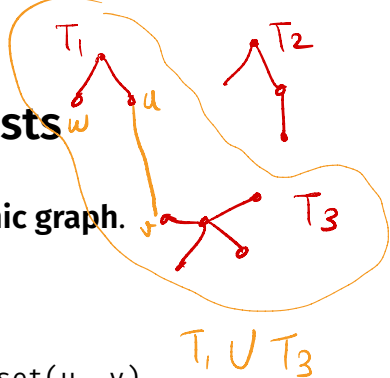
```
>>> # create a DSF with {{0}, {1}, {2}, {3}, {4}, {5}}
>>> dsf = DisjointSetForest([0, 1, 2, 3, 4, 5])
>>> dsf.union(0, 3)    { {0,3}, {1}, {2}, {4}, {5} }
>>> dsf.union(1, 4)    { {0,3}, {1,4}, {2}, {5} }
>>> dsf.union(3, 1)    { {0,3}, {1,4}, {2}, {5} }
>>> dsf.union(2, 5)    { {0,3}, {1,4}, {2,5} }
>>> # dsf now represents {{0, 1, 3, 4}, {2, 5}}
>>> dsf.in_same_set(0, 3)
True
>>> dsf.in_same_set(0, 2)
False
```

Disjoint Set Forests

- ▶ Operations take $\Theta(\alpha(n))$ time, where n is number of objects in collection.
- ▶ $\alpha(n)$ is the inverse Ackermann function.
- ▶ It grows very, **very** slowly.
- ▶ Essentially constant time.

Disjoint Set Forests

- ▶ Can be used to keep track of CCs of a **dynamic graph**.
- ▶ Nodes of CCs are disjoint sets.
 - ▶ Add an edge (u, v) : `.union(u, v)`
 - ▶ Check if u and v are connected: `.in_same_set(u, v)`
- ▶ To check if u, v are already connected:
 - ▶ BFS/DFS: $\Theta(V)$ each time.
 - ▶ DSF: $\Theta(\alpha(V))$ each time (essentially $\Theta(1)$).



Kruskal's Algorithm

```
def kruskal(graph, weights):  
    mst = UndirectedGraph()  
  
    # place each node in its own disjoint set  
    components = DisjointSetForest(graph.nodes)  
  
    # sort edges in ascending order by weight  
    sorted_edges = sorted(graph.edges, key=weights)  
  
    for (u, v) in sorted_edges:  
        if not components.in_same_set(u, v):  
            mst.add_edge(u, v)  
            components.union(u, v)  
  
            # (optional) if mst is now a spanning tree, break  
            if len(mst.edges) == len(graph.nodes) - 1:  
                break  
  
    return mst
```

Time Complexity

```
def kruskal(graph, weights):  
    mst = UndirectedGraph()
```

```
    # place each node in its own disjoint set
```

```
    components = DisjointSetForest(graph.nodes)  $\rightarrow \theta(V)$ 
```

```
    # sort edges in ascending order by weight
```

```
    sorted_edges = sorted(graph.edges, key=weights)  $\rightarrow \theta(E \log E)$ 
```

```
    for (u, v) in sorted_edges:
```

```
        if not components.in_same_set(u, v):  $\rightarrow \theta(1)$ 
```

```
            mst.add_edge(u, v)  $\rightarrow \theta(1)$ 
```

```
            components.union(u, v)  $\rightarrow \theta(1)$ 
```

```
        # (optional) if mst is now a spanning tree, break
```

```
        if len(mst.edges) == len(graph.nodes) - 1:
```

```
            break
```

```
    return mst
```

$\theta(1)$

total: $\theta(V) +$

$\theta(E \log E)$

$= \theta(E \log E)$

assume $E = \Omega(V)$

Time Complexity

- ▶ Assume graph is connected. Then $E = \Omega(V)$.
- ▶ Kruskal's takes $\Theta(E \log E) = \Theta(E \log V)$ time.
 - ▶ Dominated by sorting the edges.
- ▶ Note: if graph disconnected, Kruskal's produces a **minimum spanning forest**.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 9 | Part 7

Kruskal v. Prim

Kruskal v. Prim

- ▶ Both algorithms for computing MSTs.
- ▶ Which is “better”?
- ▶ There's no clear winner.

Time Complexity

- ▶ Prim:
 - ▶ Binary heap: $\Theta(V \log V + E \log V)$
 - ▶ Fibonacci heap: $\Theta(V \log V + E)$
- ▶ Kruskal: $\Theta(E \log V)$
- ▶ If the graph is dense, $E = \Theta(V^2)$, and Prim's with Fibonacci heap "wins".
 - ▶ $\Theta(V^2)$ versus $\Theta(V^2 \log V)$.

Not so fast...

- ▶ Fibonacci heaps are hard to implement, high overhead.
- ▶ Prim's will be faster for very large dense graphs.
- ▶ But Kruskal's may be faster for smaller dense graphs.
- ▶ The right choice depends on your application.

Main Idea

Asymptotic time complexity isn't everything. For small inputs, the “inefficient” algorithm may beat the “efficient” one. There's also ease of implementation to consider.

CS-GY 6033

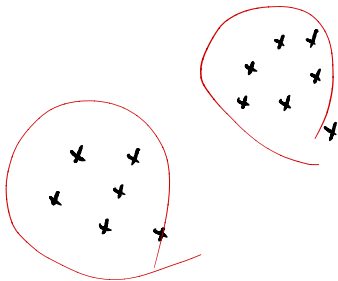
Design and Analysis of Algorithms I

Lecture 9 | Part 8

MSTs and Clustering

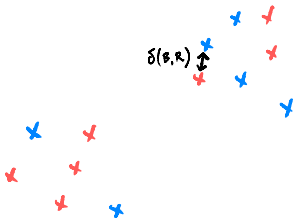
Clustering

Goal: identify the groups in data. Example:



Clustering, Formalized

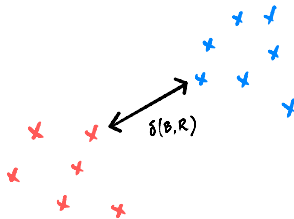
- ▶ We frame as an optimization problem.
 - ▶ GIVEN: n data points.
 - ▶ GOAL: assign color to each point (red or blue) to maximize the distance between the closest pair of red and blue points.



Bad Clustering

Clustering, Formalized

- ▶ We frame as an optimization problem.
 - ▶ GIVEN: n data points.
 - ▶ GOAL: assign color to each point (red or blue) to maximize the distance between the closest pair of red and blue points.



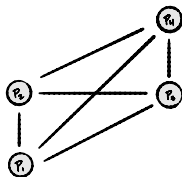
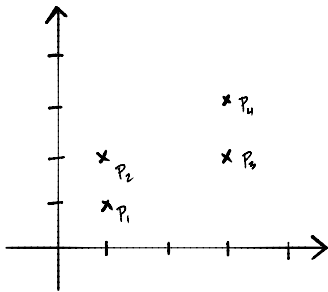
Good Clustering

Brute Force Solution

- ▶ Try all possible assignments; return best.
- ▶ If there are n data points, there are $\Theta(2^n)$ assignments.
- ▶ Exponential time; very slow. Practical only for ~ 50 data points.
- ▶ Instead, we will turn it into a graph problem.

Distance Graphs

- ▶ Given n data points, $p_1, p_2, \dots, p_n$, create complete graph with $V = \{p_1, \dots, p_n\}$.
- ▶ Set weight of edge $(p_i, p_j) = \text{dist}(p_i, p_j)$.
- ▶ The result is a weighted, undirected **distance graph**.

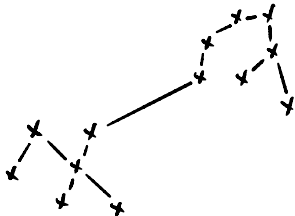


Main Idea

We can always think of a set of points in a (metric) space as a weighted distance graph. This is a **very** important idea, because it allows us to use our graph algorithms!

Clustering with MSTs

- ▶ Given n data points and a number of clusters, k :
 - ▶ Create distance graph G .
 - ▶ Run Kruskal's Algorithm on G until there are only k components.



- ▶ The resulting connected components are the **clusters**.
- ▶ This is known as **single-linkage clustering**.

Example

Single-Linkage Clustering

- ▶ Time complexity of single-linkage is determined by Kruskal's Algorithm: $\Theta(E \log E)$.
- ▶ Since distance graph is complete, $E = \Theta(V^2)$, and so

$$\Theta(E \log E) = \Theta(V^2 \log V) = \Theta(n^2 \log n)$$

- ▶ Practically, can cluster $\sim 10,000$ points.

Summary

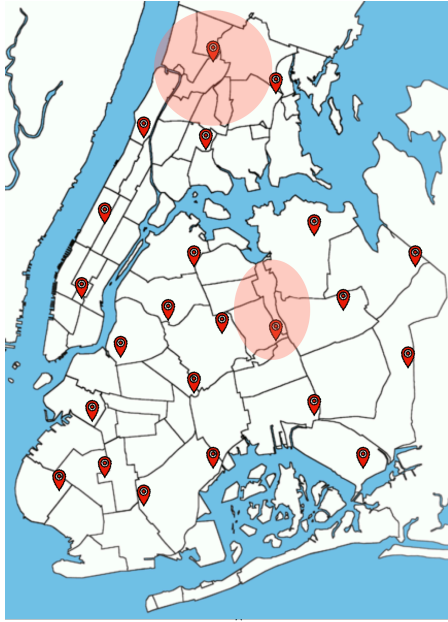
- ▶ We started the quarter with a brute force solution.
 - ▶ Took $\Theta(2^n)$ time, only feasible for a few dozen points.
- ▶ We've now reframed the problem using graph theory.
 - ▶ Now only $\Theta(n^2 \log n)$ time!
 - ▶ Feasible for tens of thousands of points.

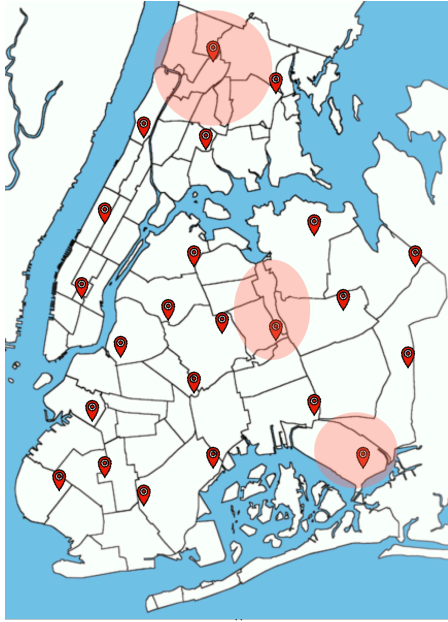
Problem

- ▶ A city's government is looking to improve its emergency services (fire stations) to ensure maximum coverage of the city's neighborhoods.
- ▶ **Constraint:** Due to budget constraints, they can only afford to set up a limited number of new facilities.
- ▶ **Goal:** Identify locations for the new fire stations that maximize the number of neighborhoods within a reasonable response time (e.g., within 10 minutes of travel time).









Abstracting the problem

- ▶ Graphs are a popular way of visualizing the problem
- ▶ Computers: adjacency matrix, adjacency list, dictionary,...

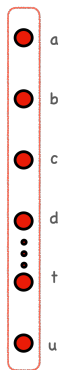
Abstracting the problem

- ▶ Nodes:
 - ▶ Each neighbourhood is represented by a blue node
 - ▶ Each facility location is represented by a red node

Neighbourhoods

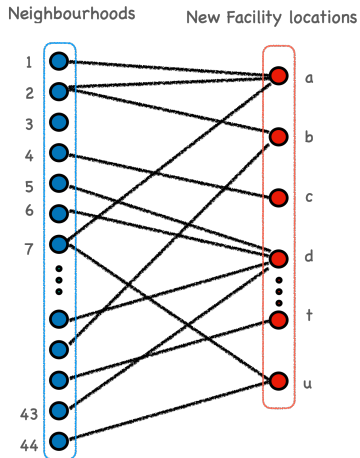


New Facility locations



Abstracting the problem

- Edges:
 - Connect a facility to a neighbourhood if it is within the response time

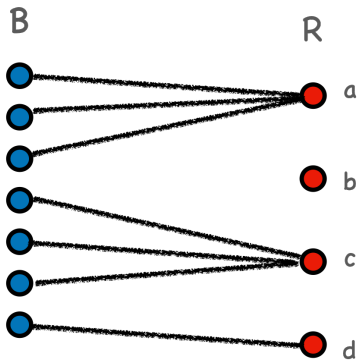


Maximum Coverage

- ▶ **Input:** Graph $G = (B, R, E)$, and budget k .
- ▶ **Objective:** find a subset of red nodes in R of size at most k that covers maximum number of blue nodes in B .

Example 1

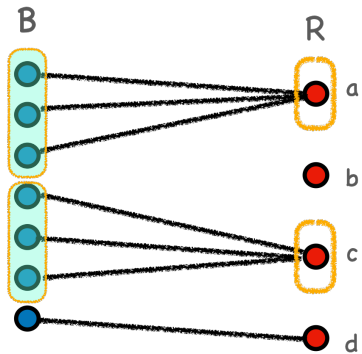
- What is the largest coverage achieved by selecting 2 red nodes ?



Example 1

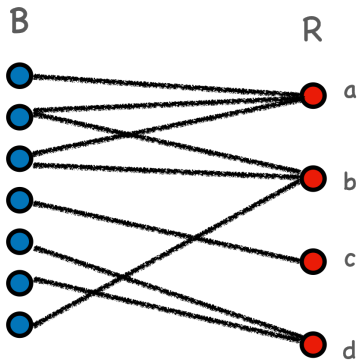
- What is the largest coverage achieved by selecting 2 red nodes ?

6

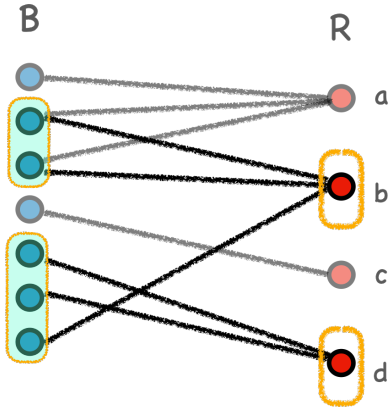


Example 2

- What is the largest coverage achieved by selecting 2 red nodes ?

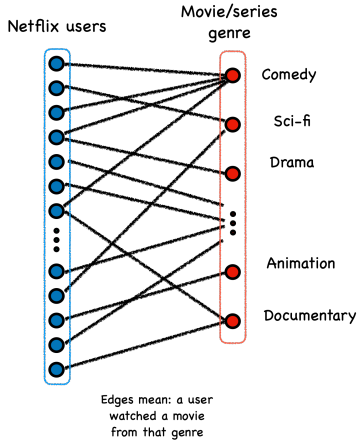


Example 2

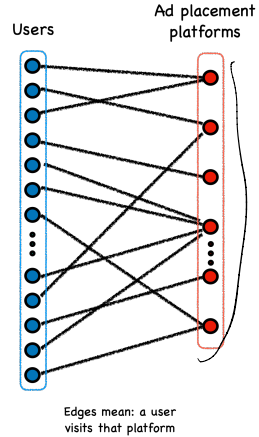


It covers 5 blue nodes

Problem: Pick 4 genres with maximum number of viewers.



Problem: Pick 6 platforms to place your ad that gets the most visit.



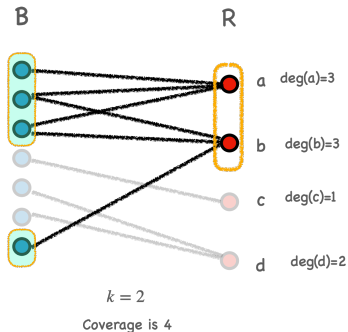
First attempt (brute force)

- ▶ Try all subsets of red nodes of size k .
- ▶ For each of them compute how many blue nodes it covers
- ▶ Output the best one.
- ▶ **Guarantee:** it finds the best solution!
- ▶ There are $\binom{n}{k}$ subsets, check the coverage of each subset.
- ▶ Too slow (e.g. 21^5 is almost 4 million cases.)

Second attempt (greedy)

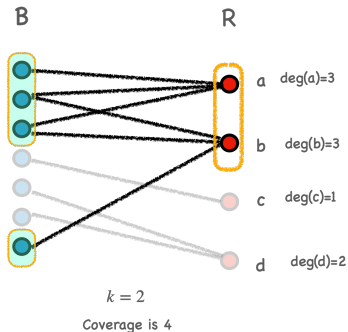
► **Algorithm:**

- Pick k red nodes with maximum degree.
- This is a super fast algorithm.



Second attempt (greedy)

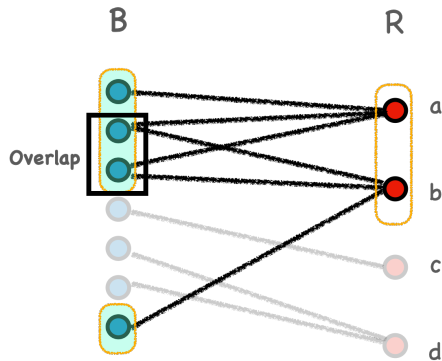
- ▶ **Algorithm:**
 - ▶ Pick k red nodes with maximum degree.
- ▶ This is a super fast algorithm.
- ▶ Drawbacks ?



Second attempt (greedy)

- ▶ **Algorithm:**

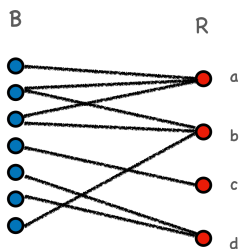
- ▶ Pick k red nodes with maximum degree.
- ▶ This is a super fast algorithm.
- ▶ Drawbacks ?
 - ▶ Treats nodes independently, ignores overlaps.



$k = 2$

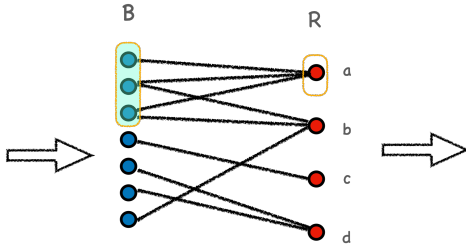
Coverage is 4

Avoiding overlaps



$$S_0 = \emptyset$$

Start with an empty set

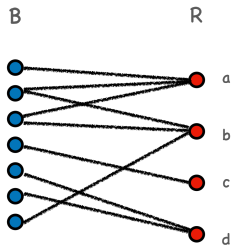


$$S_1 = \{a\}$$

Select a, it has the maximum coverage

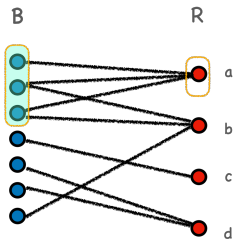
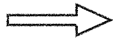
- Do not care about the nodes that are already covered.
- Remove them from consideration

Avoiding overlaps



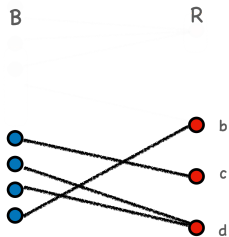
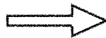
$$S_0 = \emptyset$$

Start with an empty set



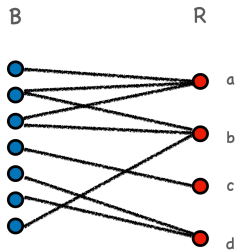
$$S_1 = \{a\}$$

Select a, it has the maximum coverage



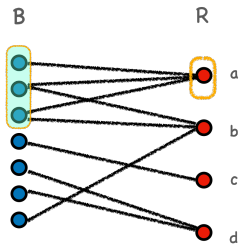
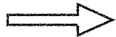
Find the best node in this "new" graph

Avoiding overlaps



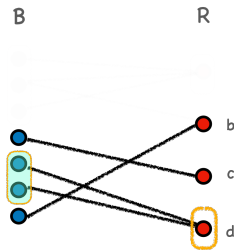
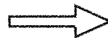
$$S_0 = \emptyset$$

Start with an empty set



$$S_1 = \{a\}$$

Select a, it has the maximum coverage



$$S_1 = \{a, d\}$$

The greedy choice is d,

Third attempt (adaptive greedy algorithm)

Start with an empty set $S = \emptyset$ # Selected red node

for $i = 1$ to k : # Iteratively add an element to S

a node that covers the maximum number of uncovered neighbourhoods

$v \leftarrow$ node with maximum coverage in G

update S

$S = S + v$

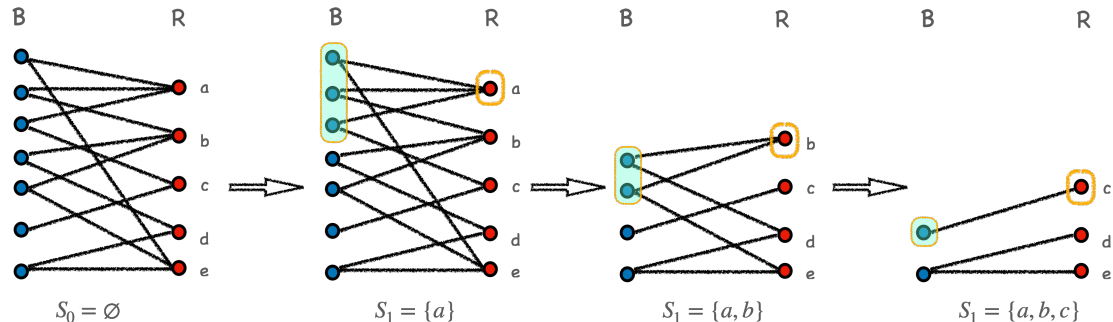
update G by removing v and its neighbours $N(v)$ and their incident edges

$G = G - (v + N(v))$

return S

Is greedy optimal?

Example: greedy is not optimal



Start with an empty set

The greedy coverage is 6.

Optimal coverage is 7, $S = \{b, c, e\}$

Question

- ▶ Does adaptive greedy work for other instances?
- ▶ **Observation:** after each iteration, the number of covered nodes increases.
- ▶ **Main question:** figure out the improvement gained at each iteration.

Main idea

- ▶ **Notation:** for every subset S of red nodes, let $f(S)$ denote the number of blue nodes it covers.
- ▶ **Main idea:** keep track of the improvement after each iteration.
- ▶ **Loop invariant:** let S_i be the set of selected nodes at each iteration i , then

$$f(S_{i+1}) - f(S_i) \geq \frac{1}{k} (OPT - f(S_i))$$

Minimum improvement at
each iteration

Proof

Suppose the loop invariant holds:

$$\begin{array}{ll} 1) & OPT - f(S_i) \leq (1 - \frac{1}{k}) (OPT - f(S_{i-1})) \quad \Rightarrow \quad \text{Follows from the loop invariant} \\ & \quad \quad \quad \downarrow \\ 2) & \leq (1 - \frac{1}{k}) (1 - \frac{1}{k}) (OPT - f(S_{i-2})) \quad \Rightarrow \quad \text{Inductively apply the loop invariant} \\ & \quad \quad \quad \downarrow \\ 3) & \leq (1 - \frac{1}{k}) (1 - \frac{1}{k}) (1 - \frac{1}{k}) (OPT - f(S_{i-3})) \quad \Rightarrow \quad \text{Inductively apply the loop invariant} \\ & \quad \quad \quad \vdots \\ 4) & \leq (1 - \frac{1}{k})^i (OPT - f(S_0)) \quad \Rightarrow \quad \text{Recall } f(S_0) = f(\emptyset) = 0 \\ 5) & = (1 - \frac{1}{k})^i \cdot OPT \end{array}$$

Proof

Suppose the loop invariant holds:

$$\begin{aligned} 1) \quad & OPT - f(S_i) \leq \left(1 - \frac{1}{k}\right) (OPT - f(S_{i-1})) && \Rightarrow \text{Follows from the loop invariant} \\ 2) \quad & \leq \left(1 - \frac{1}{k}\right) \left(1 - \frac{1}{k}\right) (OPT - f(S_{i-2})) && \Rightarrow \text{Inductively apply the loop invariant} \\ 3) \quad & \leq \left(1 - \frac{1}{k}\right) \left(1 - \frac{1}{k}\right) \left(1 - \frac{1}{k}\right) (OPT - f(S_{i-3})) && \Rightarrow \text{Inductively apply the loop invariant} \\ & \vdots \\ 4) \quad & \leq \left(1 - \frac{1}{k}\right)^i (OPT - f(S_0)) && \Rightarrow \text{Recall } f(S_0) = f(\emptyset) = 0 \\ 5) \quad & = \left(1 - \frac{1}{k}\right)^i \cdot OPT \end{aligned}$$

- By rearranging and setting $i = k$

$$f(S_k) \geq \left(1 - \left(1 - \frac{1}{k}\right)^k\right) OPT$$

What does it mean?

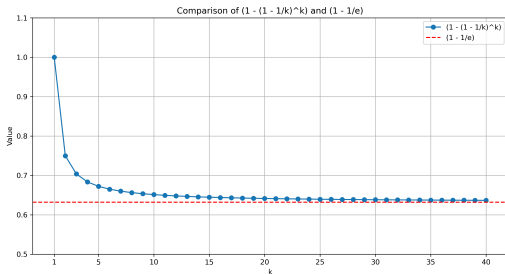
- ▶ We showed $f(S_k) \geq (1 - (1 - \frac{1}{k})^k) \text{OPT}$
- ▶ For $k = 1$, it means $f(S_1) \geq \text{OPT}$

What does it mean?

- ▶ We showed $f(S_k) \geq (1 - (1 - \frac{1}{k})^k) \text{OPT}$
- ▶ For $k = 1$, it means $f(S_1) \geq \text{OPT}$
- ▶ For $k = 2$, it means $f(S_2) \geq 0.75 \cdot \text{OPT}$

What does it mean?

- ▶ We showed $f(S_k) \geq (1 - (1 - \frac{1}{k})^k) \text{OPT}$
- ▶ For $k = 1$, it means $f(S_1) \geq \text{OPT}$
- ▶ For $k = 2$, it means $f(S_2) \geq 0.75 \cdot \text{OPT}$
- ▶ For $k = 5$, it means $f(S_5) \geq 0.672 \cdot \text{OPT}$
- ▶ As k grows, it converges to 0.632



What does it mean?

- ▶ This greedy approach is very popular:
 - ▶ Influence maximization, marketing, ad placement, recommender system
- ▶ Greedy is fast, but still slow for large datasets: Netflix, Meta, Amazon have millions of users
- ▶ Privacy and fairness:
 - ▶ How can we guarantee user privacy? Fairness ?

